

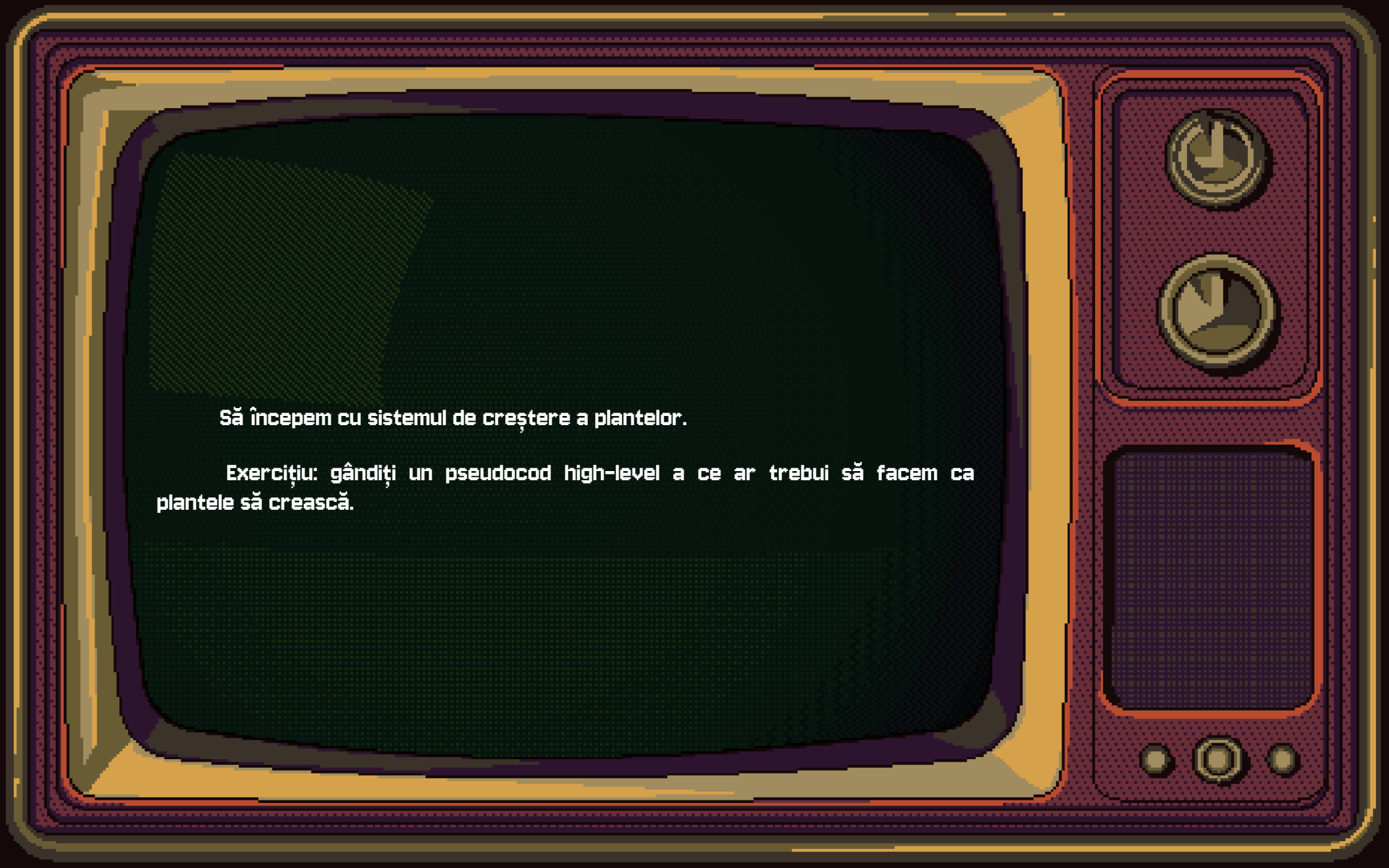
Prezentarea 5

Prof. Coordonator: Ș.I.dr.ing.Stelian-Nicolae NICOLA

Mentor: Pătrașcu Hary

The image shows a pixelated, retro-style interface. On the left is a large, dark, rounded rectangular area that serves as a main display or workspace. To the right of this area is a vertical control panel. This panel contains two circular buttons at the top, each with a different icon (one looks like a power button, the other like a play button). Below these is a larger rectangular area with a grid pattern, possibly a menu or a list. At the bottom of the control panel are three small circular buttons. The entire interface is framed by a thick, multi-layered border with a yellow and orange color scheme.

În prezentarea anterioară am reușit să implementăm funcționalitatea de plantare, dar mai există aspecte care trebuie implementate. În momentul actual plantele nu pot crește, jucătorul nu le poate colecta și nu poate selecta tipul de sămânță pe care vrea să o planteze.

The image shows a pixelated, retro-style interface. On the left is a large, dark, rounded rectangular area that serves as a screen. To the right of this screen is a control panel. The panel has a textured, reddish-brown background. At the top of the panel are two circular buttons, each with a white icon. Below these is a larger rectangular area with a grid of small squares, resembling a digital display or a menu. At the bottom of the panel are three small circular buttons. The entire interface is framed by a thick, multi-layered border in shades of yellow, orange, and red.

Să începem cu sistemul de creștere a plantelor.

Exercițiu: gândiți un pseudocod high-level a ce ar trebui să facem ca plantele să crească.



1. Un sistem de timp

2. Când se schimbă ziua, să verific că tile-ul pe care se află planta este ud

Sistemul de timp(Timers)

Godot dispune de un nod numit Timer care este folosit pentru a implementa mecanici care necesită temporizare, de exemplu trecerea zilelor, cooldown-uri pentru abilități, spawnarea inamicilor la anumite intervale de timp, etc.



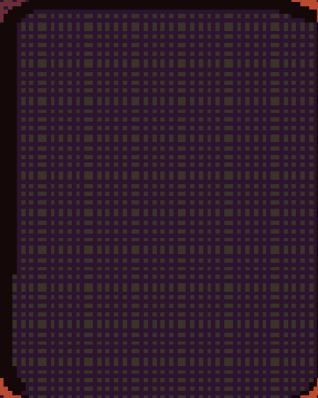
Exercițiu: Creați o scenă numită TimeManager cu nodul părinte Timer. Atașați un script acestei scene și adăugați scena în scena Level.

- Level
 - Environment
 - OceanLayer
 - GrassLayer
 - DirtLayer
 - WaterDirtLayer
 - Plants
 - Player
 - TimeManager

Filter Scripts

- level.gd
- time_manager.gd

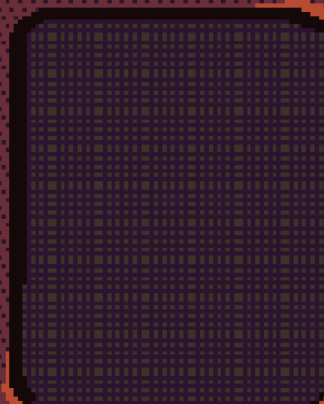
```
1 class_name TimeManager extends Timer
2
3 func _ready() -> void:
4     pass
5
```



Când selectați nodul timer puteți vizualiza toate proprietățile pe care le are. În tabul signals o să găsiți semnalul timeout care este foarte util în multe cazuri. În cazul nostru îl vom folosi pentru a declanșa anumite evenimente când se schimbă ziua. În funcție de wait time-ul pe care îl setăm, semnalul timeout se va declanșa după ce timer-ul a numărat până la waittime. De exemplu dacă setăm waittime-ul la 5 secunde, timeout se va declanșa după 5 secunde.

Exercițiu: setați proprietatea waittime la 5 și autostart pe true din editor.

Timerele pot fi create și din cod.





Acum că avem timer-ul, ar trebui să folosim semnalul de timeout.

Exercițiu: conectați semnalul de timeout la scriptul level.



```
18  
→ 19 > func _on_player_tool_used(tool: Player.Tool, pos: Vector2) -> void: ⌵  
58  
59  
→ 60 > func _on_time_manager_timeout() -> void:  
61 > | pass # Replace with function body.  
62
```

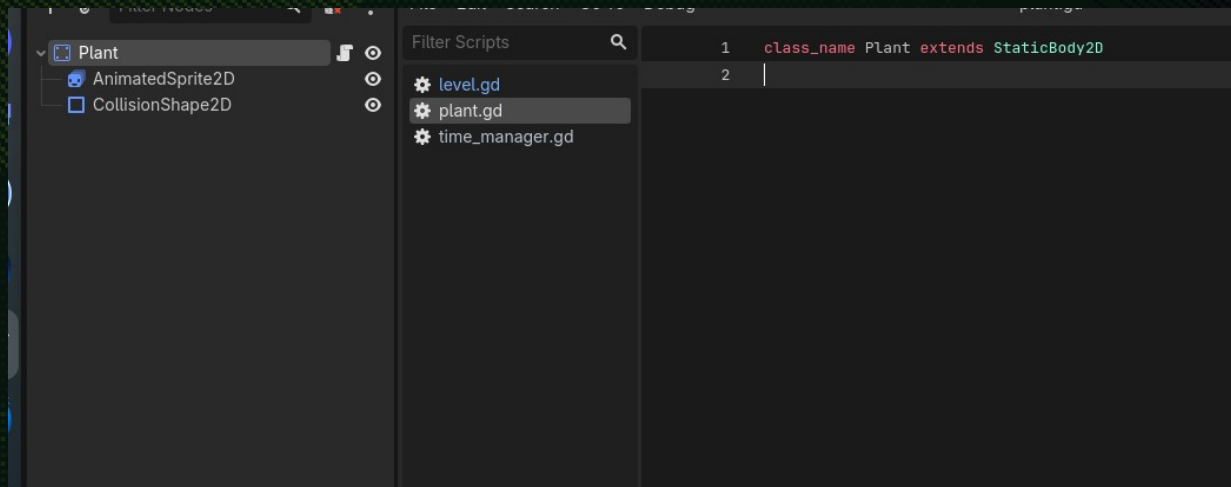
methods 🔍 ↕

yer_tool_used

e_manager_timeout



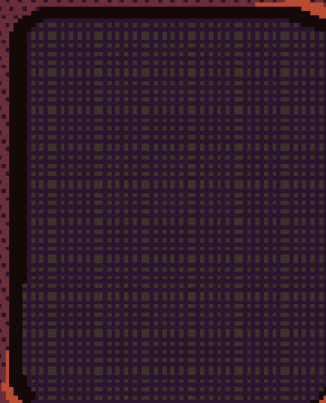
Acum că avem semnalul de timeout, ar trebui să verificăm plantele. În primul rând, să adăugăm un script la scena Plant.

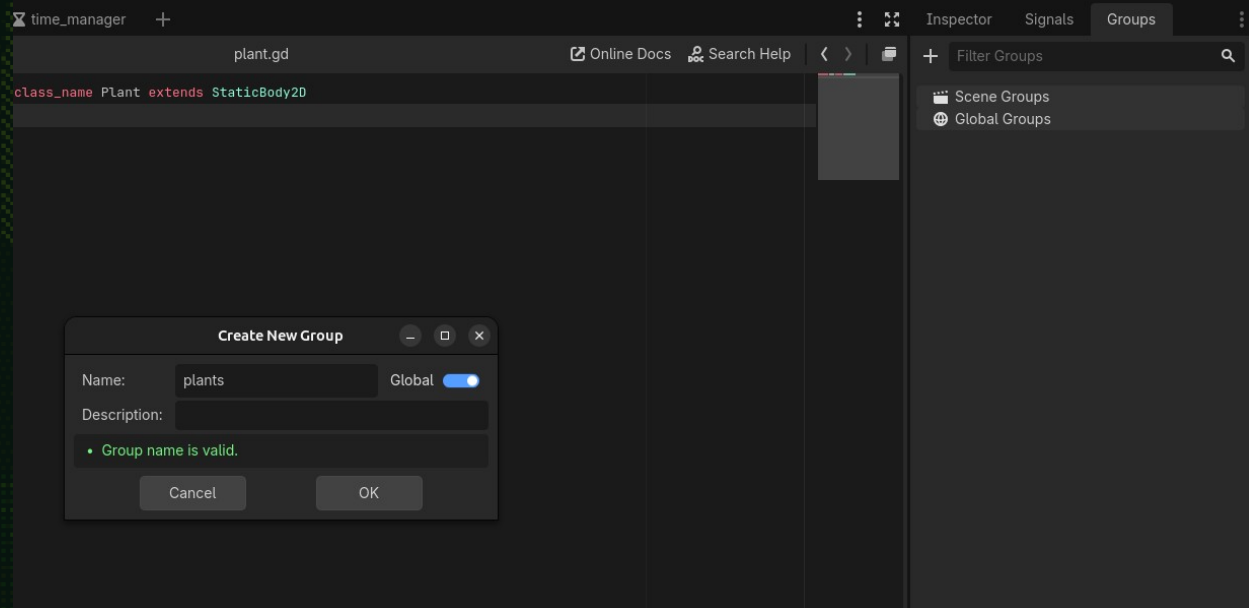


Grupuri de noduri(groups)

În Godot există o modalitate de a grupa noduri de un fel pentru a avea acces mai ușor la ele. În scena plant dacă vă uitați în dreapta o să vedeți taburile inspector, signals și groups.

Pentru a crea un grup mergeți în tabul groups și apăsați pe iconița '+' apoi dați un nume grupului, de exemplu plants și faceți grupul global pentru a putea fi accesat de către alte scripturi.







Acum din scriptul level când avem un timeout putem accesa toate plantele folosind acest grup și să apelăm o metodă de creștere a plantei de exemplu. Deoarece am făcut setările din editor, fiecare plantă pe care o instanțiem va fi adăugată automat în grupul de plante.

Exercițiu: accesați documentația Godot și încercați să găsiți o metodă care să vă dea toate nodurile dintr-un grup.



Metoda `get_nodes_in_group('nume_grup')` ne va da toate nodurile dintr-un grup. De asemenea există și metoda `call_group` care ne permite să apelăm o funcție pe toate nodurile dintr-un anumit grup.

Exercițiu: creați o metodă numită `grow_plant` în script-ul `plant.gd` și folosind metoda `call_group` apelați acea metodă din script-ul `level.gd` când aveți un timeout.



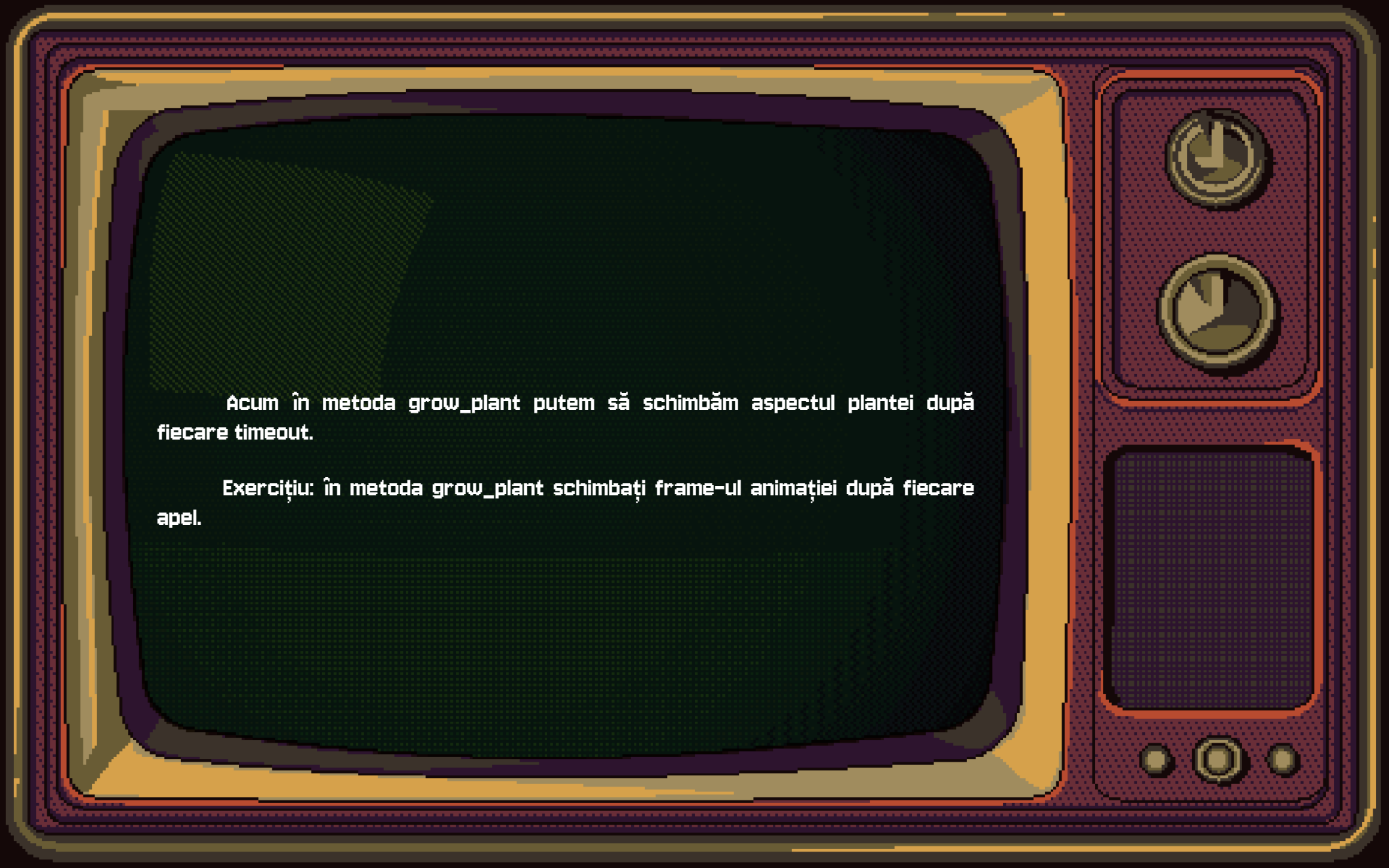
```
1  class_name Plant extends StaticBody2D
2
3
4  ▼ func grow_plant() -> void:
5    >| print("plant has grown")|
6    void print(...args: Array)
```

59

60 ▼ `func _on_time_manager_timeout() -> void:`

61 `> get_tree().call_group("plants", "grow_plant")`

62



Acum în metoda `grow_plant` putem să schimbăm aspectul plantei după fiecare timeout.

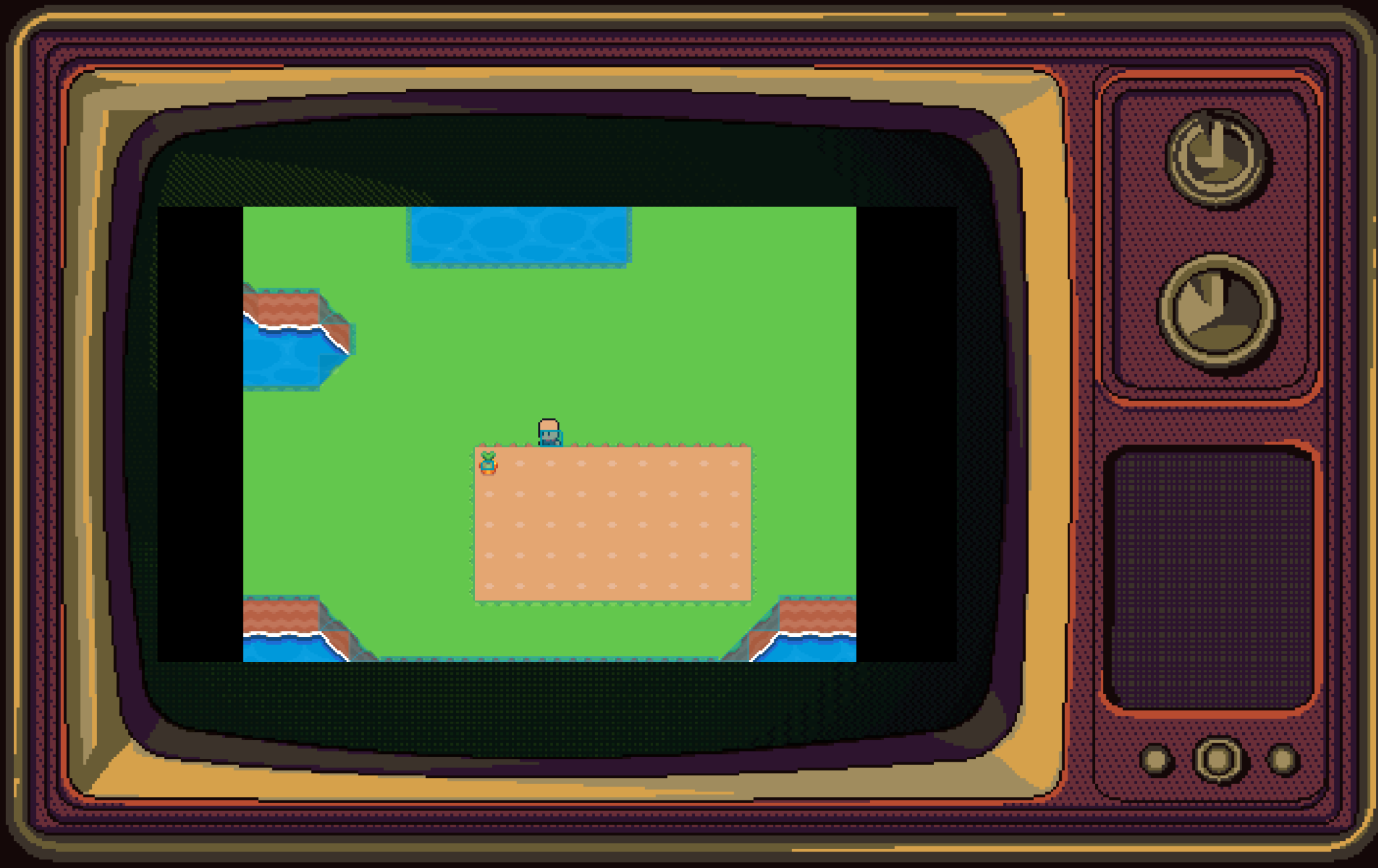
Exercițiu: în metoda `grow_plant` schimbați frame-ul animației după fiecare apel.



În primul rând avem nevoie de o referință la nodul care are frame-urile anume `animatedsprite2d`. Apoi putem accesa proprietatea `frame` și să o incrementăm.



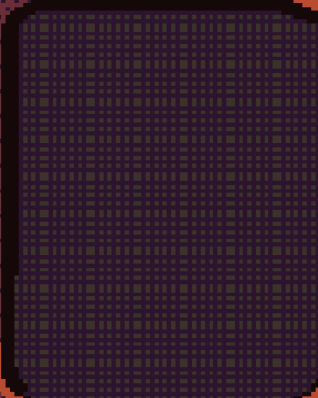
```
1  class_name Plant extends StaticBody2D
2
3  @onready var animated_sprite_2d: AnimatedSprite2D = $AnimatedSprite2D
4
5  func grow_plant() -> void:
6      >| print("plant has grown")
7      >| animated_sprite_2d.frame = animated_sprite_2d.frame + 1|
8
```

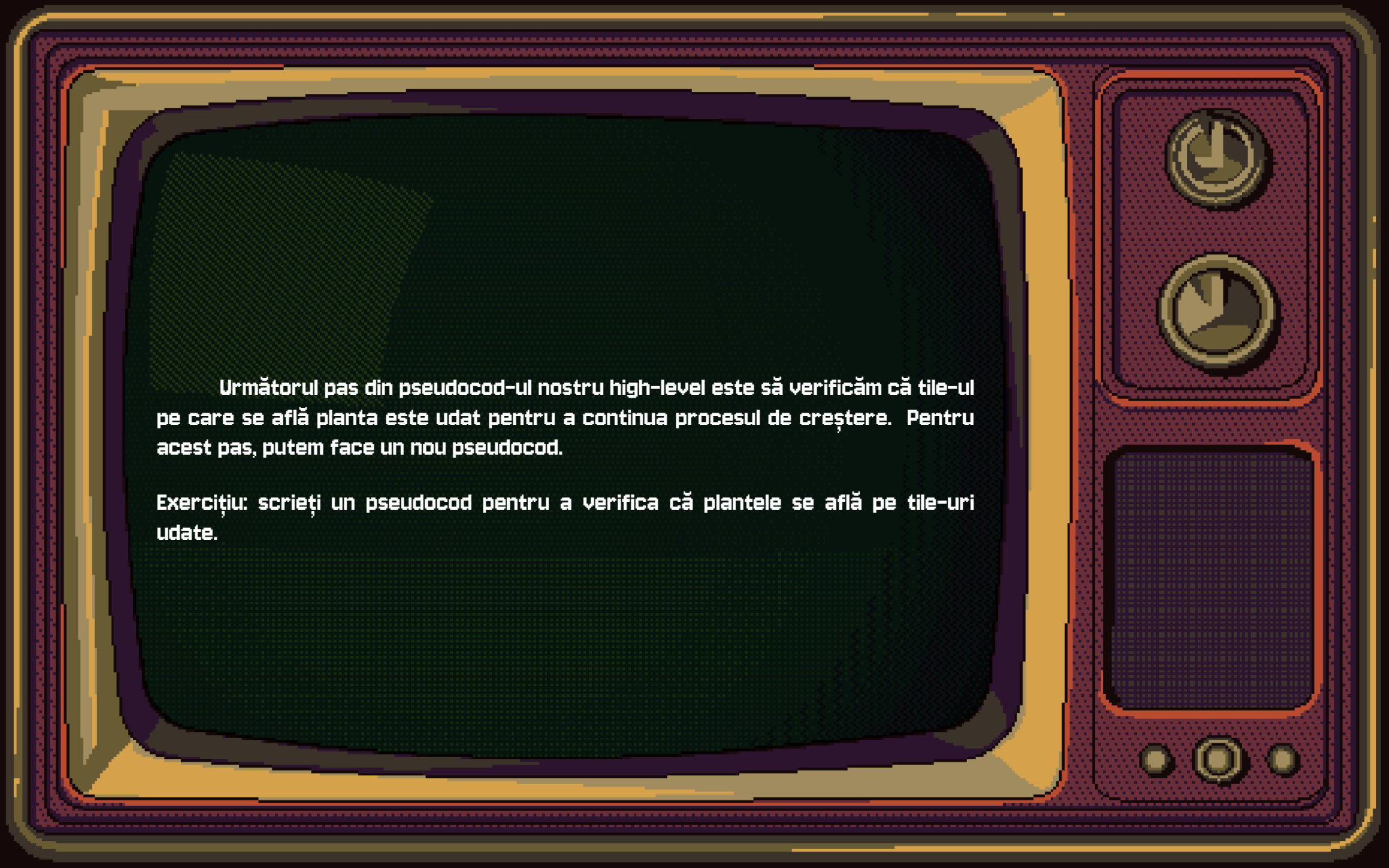




Mai trebuie să verificăm că planta nu a ajuns la ultimul frame pentru a
continua numărătoarea.

```
1  class_name Plant extends StaticBody2D
2
3  @onready var animated_sprite_2d: AnimatedSprite2D = $AnimatedSprite2D
4
5  func grow_plant() -> void:
6      >| # check the frame is less than the frame count and then increment the frame|
7      >| if animated_sprite_2d.frame < animated_sprite_2d.sprite_frames.get_frame_count("carrot"):
8          >|     animated_sprite_2d.frame = animated_sprite_2d.frame + 1
9          >|     print("plant has grown")
10
```





Următorul pas din pseudocod-ul nostru high-level este să verificăm că tile-ul pe care se află planta este udat pentru a continua procesul de creștere. Pentru acest pas, putem face un nou pseudocod.

Exercițiu: scrieți un pseudocod pentru a verifica că plantele se află pe tile-uri udate.

```
9 # 1. trebuie să verificăm fiecare plantă → putem folosi un for loop
8 # 2. pentru a face rost de toate plantele din grup avem o metodă → get_tree().get_nodes_in_group("plants")
7 # 3. trebuie la fiecare plantă să verificăm că se află pe un tile de dirt udat → verificăm pe water dirt layer
6 # 4. avem o metodă pe care am folosit-o anterior, get_cell_tile_data() care primește o poziție și returnează null dacă nu există
5 # un tile la acea poziție pe acel layer
4 # 5. știind că avem nevoie de poziția de grid pentru acea metodă → ar trebui să salvăm în script-ul plantei pe care o spawnăm
3 # poziția la care a fost spawnată(grid position-ul)
2 # 6. dacă la poziția aceea se află un tile de apă udat → apelăm plant.grow_plant() pe planta curentă din for loop
1 # → altfel, nu facem nimic la acea iterație
```



Exercițiu: bazat pe pseudocodul scris de voi sau cel scris de mine, încercați să implementați pas cu pas din limbaj natural în cod.

```
8 func _on_time_manager_timeout() → void:
7     # 2. pentru a face rost de toate plantele din grup avem o metodă → get_tree().get_nodes_in_group("plants")
6     var plant_nodes: Array = get_tree().get_nodes_in_group("plants")
5     # 1. trebuie să verificăm fiecare plantă → putem folosi un for loop
4     for plant: Plant in plant_nodes:
3         pass
```

```
9 func _on_time_manager_timeout() -> void:
8 # 2. pentru a face rost de toate plantele din grup avem o metodă -> get_tree().get_nodes_in_group("plants")
7 var plant_nodes: Array = get_tree().get_nodes_in_group("plants")
6 # 1. trebuie să verificăm fiecare plantă -> putem folosi un for loop
5 for plant: Plant in plant_nodes:
4 # 3. trebuie la fiecare plantă să verificăm că se află pe un tile de dirt udat -> verificăm pe water dirt layer
3 # 4. avem o metodă pe care am folosit-o anterior, get_cell_tile_data() care primește o poziție și returnează null dacă nu există
2 # un tile la acea poziție pe acel layer
1 if water_dirt_layer.get_cell_tile_data(plant.grid_pos):
72 plant.grow_plant()
```

```
14 Player.Tool.CARRY:
13     # daca tile-ul pe care incerc sa il vd este de tip dirt
12     var dirt_data := dirt_layer.get_cell_tile_data(grid_pos)
11     # altfel inchei
10     if not dirt_data:
9         return
8
7     if not grid_pos in planted_cells:
6         var plant = PLANT.instantiate() as Plant
5         plants.add_child(plant)
4         plant.position = (grid_pos * TILE_SIZE) + Vector2i(TILE_SIZE / 2, TILE_SIZE / 3)
3         # 5. știind că avem nevoie de poziția de grid pentru acea metodă → ar trebui să salvăm în script-ul plantei pe care o spawnăm
2         # poziția la care a fost spawnată(grid position-ul)
1         plant.grid_pos = grid_pos
60        planted_cells.append(grid_pos)
1
```

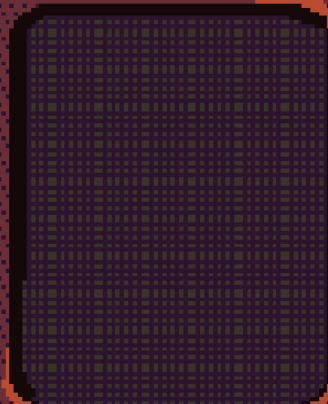

Am adăugat variabila `grid_pos` în script-ul `plant.gd`

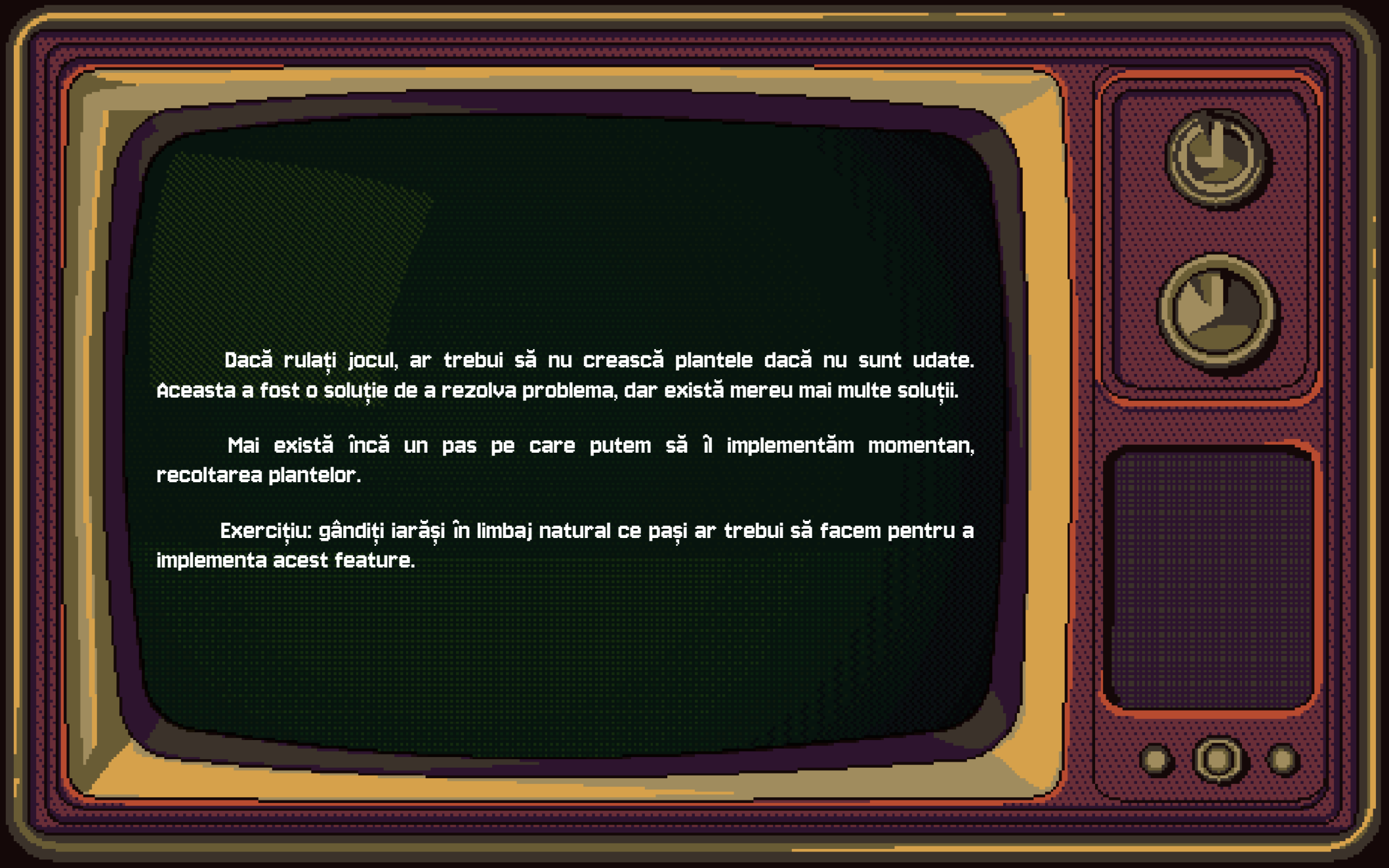
```
1 class_name Plant extends StaticBody2D
1
2 @onready var animated_sprite_2d: AnimatedSprite2D = $AnimatedSprite2D
3 var grid_pos: Vector2 = Vector2.ZERO
4
5 func grow_plant() → void:
6     # check the frame is less than the frame count and then increment the frame
7     if animated_sprite_2d.frame < animated_sprite_2d.sprite_frames.get_frame_count("carrot"):
8         animated_sprite_2d.frame = animated_sprite_2d.frame + 1
9         print("plant has grown")
```

```

11 func _on_time_manager_timeout() -> void:
10 # 2. pentru a face rost de toate plantele din grup avem o metodă -> get_tree().get_nodes_in_group("plants")
9   var plant_nodes: Array = get_tree().get_nodes_in_group("plants")
8   # 1. trebuie să verificăm fiecare plantă -> putem folosi un for loop
7   for plant: Plant in plant_nodes:
6       # 3. trebuie la fiecare plantă să verificăm că se află pe un tile de dirt udat -> verificăm pe water dirt layer
5       # 4. avem o metodă pe care am folosit-o anterior, get_cell_tile_data() care primește o poziție și returnează null dacă nu există
4       # un tile la acea poziție pe acel layer
3       if water_dirt_layer.get_cell_tile_data(plant.grid_pos):
2           # 6. dacă la poziția aceea se află un tile de apă udat -> apelăm plant.grow_plant() pe planta curentă din for loop
1           # -> altfel, nu facem nimic la acea iterație
74         plant.grow_plant()
1

```





Dacă rulați jocul, ar trebui să nu crească plantele dacă nu sunt udate. Aceasta a fost o soluție de a rezolva problema, dar există mereu mai multe soluții.

Mai există încă un pas pe care putem să îl implementăm momentan, recoltarea plantelor.

Exercițiu: gândiți iarăși în limbaj natural ce pași ar trebui să facem pentru a implementa acest feature.

```
11 # 1. In primul rând să alegem un tool care să se ocupe de recoltare, asset pack-ul folosit
10 # nu dispune de un tool dedicat pentru această acțiune, astfel, putem folosi lopata(dig)
9 # 2. trebuie să verificăm că suntem în cazul dig(pentru lopată) și că există o plantă la
8 # coordonatele alese
7 # 3. avem deja un array care stochează coordonatele plantelor unde putem verifica dacă
6 # la coordonatele alese de tool(grid_pos) există o plantă
5 # 4. ar trebui acces la obiectul plantă pentru a îl putea șterge din memorie când îl colectăm
4 # 5. putem modifica array-ul într-un dicționar care stochează pozițiile plantelor și în plus, o referință la
3 # nodul plantă respectiv
2 # 6. dacă în dicționar avem poziția grid-ului, putem șterge planta
1 # 7. opțional putem adăuga niște efecte vizuale, în loc să ștergem planta instant, îi schimbăm poziția și rămâne pe pământ
104 # apoi când jucătorul se aproprie de ea, va fi colectată(la fel ca în stardew valley)
```

```

15 func _on_player_tool_used(tool: Player.Tool, pos: Vector2) → void:
14     var grid_pos: Vector2i = Vector2i(floor(pos.x / TILE_SIZE), floor(pos.y / TILE_SIZE))
13     print(grid_pos)
12     print(pos)
11     match tool:
10         # 1. În primul rând să alegem un tool care să se ocupe de recoltare, asset pack-ul folosit
9         # nu dispune de un tool dedicat pentru această acțiune, astfel, putem folosi lopata(dig)
8         Player.Tool.DIG:
7             dirt_layer.set_cells_terrain_connect([grid_pos], 0, 0, false)
6         thesis
5             # 2. trebuie să intrăm în cazul dig(pentru lopată) și verificăm că există o plantă la
            # coordonatele alese

```

```
5 # 3. avem deja un array care stochează coordonatele plantelor unde putem verifica dacă  
4 # la coordonatele alese de tool(grid_pos) există o plantă  
3 # 4. ar trebui acces la obiectul plantă pentru a îl putea șterge din memorie când îl colectăm  
2 # 5. putem modifica array-ul într-un dicționar care stochează pozițiile plantelor și în plus, o referință la  
1 # nodul plantă respectiv  
22 var planted_cells: Dictionary[Vector2i, Plant] = {}
```



```

13 func _on_player_tool_used(tool: Player.Tool, pos: Vector2) → void:
14     var grid_pos: Vector2i = Vector2i(floor(pos.x / TILE_SIZE), floor(pos.y / TILE_SIZE))
15     match tool:
16         # 1. În primul rând să alegem un tool care să se ocupe de recoltare, asset pack-ul folosit
17         # nu dispune de un tool dedicat pentru această acțiune, astfel, putem folosi lopata(dig)
18         Player.Tool.DIG:
19             dirt_layer.set_cells_terrain_connect([grid_pos], 0, 0, false)
20             # 2. trebuie să intrăm în cazul dig(pentru lopată) și verificăm că există o plantă la
21             # coordonatele alese
22             if planted_cells.has(grid_pos):
23                 # 6. dacă în dicționar avem poziția grid-ului, putem șterge planta
24                 planted_cells[grid_pos].queue_free()
25                 # ștergem din dicționar cheia plantei
26                 planted_cells.erase(grid_pos)

```

Când schimbați din Array în dicționar, o să primiți o eroare, asta datorită faptului că dicționarele nu dispun de metoda `append` a array-urilor. În cazul `carry` ați adăugat coordonatele folosind metoda `append`.

Exercițiu: încercați să modificați codul, știm că am modificat variabila `planted_cells` într-un dicționar cu cheia `Vector2i`(o poziție) și valoarea `plant`. Căutați în documentație cum se folosește un dicționar și cum se asignează valori dicționarelor.

```

16 Player.Tool.CARRY:
15     # daca tile-ul pe care incerc sa il vd este de tip dirt
14     var dirt_data := dirt_layer.get_cell_tile_data(grid_pos)
13     # altfel inchei
12     if not dirt_data:
11         return
10
9     if not grid_pos in planted_cells:
8         var plant = PLANT.instantiate() as Plant
7         plants.add_child(plant)
6         plant.position = (grid_pos * TILE_SIZE) + Vector2i(TILE_SIZE / 2, TILE_SIZE / 3)
5         # 5. stiind ca avem nevoie de pozitia de grid pentru acea metoda -> ar trebui sa salvam in script-ul plantei pe care o spawnam
4         # pozitia la care a fost spawnata(grid position-ul)
3         plant.grid_pos = grid_pos
2         # dictionarele nu dispun de metoda append, trebuie sa modificam cum adaugam planta
1         # planted_cells.append(grid_pos)
74        planted_cells[grid_pos] = plant

```




Când rulați jocul, dacă plantați și apoi selectați lopata veți putea recolta
acea plantă.

La momentul actual jucătorul nu va primi nici o resursă, deci urmează să
implementăm acest feature.

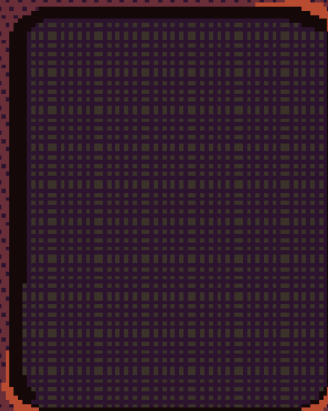
Exercițiu: gândiți din nou în limbaj natural ce ar trebui să facem astfel încât
jucătorul să primească o resursă când recoltează planta.

5 # 1. când recoltăm planta ar trebui să memorăm că jucătorul va primi acea resursă
4 # 2. putem crea un dicționar care să aibă cheia numele plantei și valoarea va fi cantitatea pe care o deține jucătorul
3 # 3. trebuie să comunicăm de la plantă la player această schimbare → o variantă ar fi să creăm o clasă globală
2 # care să conțină toate lucrurile pe care le deține jucătorul și alte proprietăți
1 # 4. când planta este ștearsă putem accesa o metodă în clasa globală care primește tipul plantei
99 # 5. în acea metodă am putea actualiza numărul de acel tip de plantă pe care le deține jucătorul

Autoloads(clase globale)

Autoloads sunt clase pe care le putem accesa din orice altă clasă din proiectul nostru. Ați lucrat până acum cu autoload-uri, anume clasa Input. Sunt clase utile dacă avem nevoie de acces la aceeași informație din mai multe clase.

În alte limbaje de programare/ environment-uri se numesc singletons și sunt considerate un design pattern.





Exercițiu: accesați documentația din link pentru singletons și încercați să creați un singleton numit `PlayerData`.

În primul rând, să creați o scenă în folder-ul player numită `player_data` cu nodul de bază Node(cel alb) apoi atașați un script cu același nume.

Dacă intrați în `project` → `project settings` → `tabul globlals`, aici puteți adăuga scena voastră.

Aceasta va fi instanțiată când porniți jocul și va avea rolul de a conține toate obiectele jucătorului(bani, plante, etc)

Exercițiu: bazat pe pseudocod-ul scris anterior, încercați să implementați dicționarul și o metodă `add_plant` care primește ca argument un `String` și nu returnează nimic(`void`).

Căutați în documentația Godot o metodă(callback) care se apelează automat când un nod este șters din lume(scene tree). În acel callback apelați metoda creată în `PlayerData`.(din scriptul `plant.gd` deoarece planta este ștearsă din scene tree)

Sintaxa de a apela o metodă/variabilă dintr-o clasă globală este `NumeleClasei.NumeMetoda(NumeArgument)`


```

10 extends Node
9
8 # 1. când recoltăm planta ar trebui să memorăm că jucătorul va primi acea resursă
7 # 2. putem crea un dicționar care să aibă cheia numele plantei și valoarea va fi cantitatea pe care o deține jucătorul
6 var plants: Dictionary[String, int] = {
5     "carrot":0,
4 }
3
2 # 5. în acea metodă am putea actualiza numărul de acel tip de plantă pe care le deține jucătorul și valoarea va fi cantitat
1 func add_plant(plant_name: String) → void:
11     pass

```



```

14 class_name Plant extends StaticBody2D
13
12 @onready var animated_sprite_2d: AnimatedSprite2D = $AnimatedSprite2D
11 var grid_pos: Vector2 = Vector2.ZERO
10
9 func grow_plant() -> void:
8     # check the frame is less than the frame count and then increment the frame
7     if animated_sprite_2d.frame < animated_sprite_2d.sprite_frames.get_frame_count("carrot"):
6         animated_sprite_2d.frame = animated_sprite_2d.frame + 1
5         print("plant has grown")
4         plants_dictionary[String, int] = {
3             "carrot": 0,
2 # 4. când planta este ștearsă putem accesa o metodă în clasa globală care primește tipul plantei
1 func _exit_tree() -> void:
15     # lasam carrot momentan, dar vom avea si alte tipuri de plante in viitor
14     PlayerData.add_plant("carrot")
13     func add_plant(plant_name: String) -> void:
12         pass
11         # verificam ca numele plantei exista in dictionar
10         if not plants.has(plant_name):
9             # daca nu e, facem un early exit din metoda si printam
8             print("%s doesn't exist in the plants dictionary" % plant_name)
7

```



Exercițiu: creați o implementare pentru metoda `add_plant`.

```
9 # 5. in acea metoda am putea actualiza numarul de acel tip de planta pe care le detine jucatorul
10 ▼ func add_plant(plant_name: String) -> void:
11     » # verificam ca numele plantei exista in dictionar
12     ▼ » if not plants.has(plant_name):
13         » » # daca nu e, facem un early exit din metoda si printam
14         » » print("%s doesn't exist in the plants dictionary" % plant_name)
15         » » return
16     » # altfel incrementam variabila
17     » plants[plant_name] += 1
18     » # printam pentru a testa ca metoda functioneaza
19     » print("avem %s %s" % [plants[plant_name], plant_name])
20
```



Dacă rulați jocul și plantați apoi folosiți lopata pe plantă, veți vedea în consolă ca valoarea va fi printată și memorată. Spre finalul modulului 2D vom face și un sistem de salvare a jocului pentru a nu pierde tot progresul de fiecare dată când închidem jocul, momentan lăsăm valorile în memoria RAM :^)



Acum că avem tool-ul de plante relativ complet, am putea lucra la partea vizuală a plantelor, în loc să îi dăm remove plantei instant putem să creăm o animație din cod care o scoate din pământ, respectiv o animație pentru când player-ul se intersectează cu ea pentru a indica că a fost adunată de pe jos.

Tweens

Tweens sunt obiecte pe care le putem crea din cod pentru animații procedurale (animații controlate din cod). Tween-urile ne permit să accesăm proprietăți ale anumitor obiecte pe care le putem schimba în timp.

În loc să dăm delete nodului direct din script-ul level.gd, am putea să creem o metodă în script-ul plant pe care să o apelăm și să animeze nodul. Când planta este atinsă de player, putem să facem altă animație procedurală și abia atunci să ștergem nodul.

Exercițiu: În scriptul plant.gd creați o metodă numită `animate_on_ground` și în level.gd în loc să dați `queue_free` apălați acea metodă pe plantă.


```

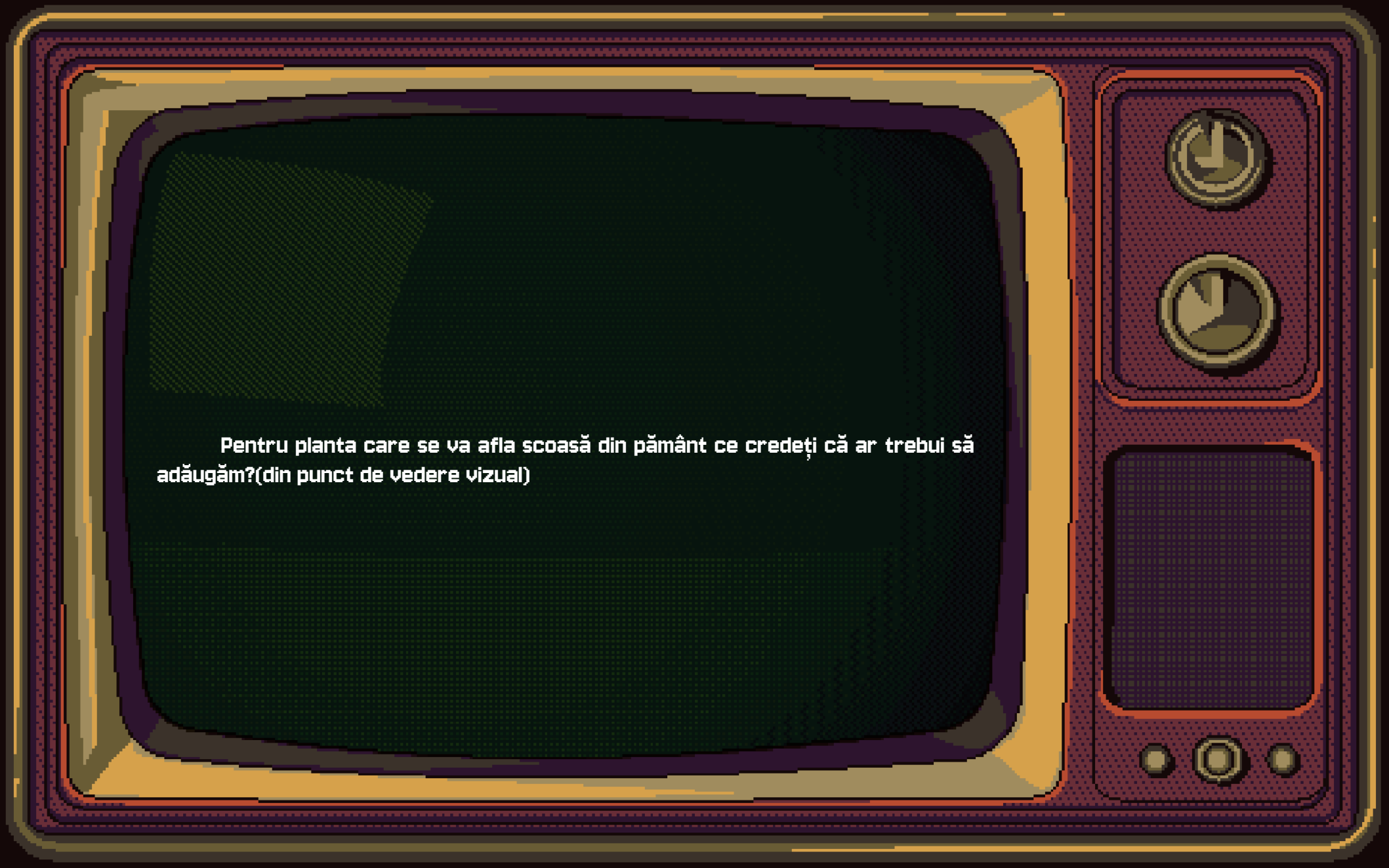
17 class_name Plant extends StaticBody2D
16
15 @onready var animated_sprite_2d: AnimatedSprite2D = $AnimatedSprite2D
14 var grid_pos: Vector2 = Vector2.ZERO
13
12 func grow_plant() -> void:
11     # check the frame is less than the frame count and then increment the frame
10     if animated_sprite_2d.frame < animated_sprite_2d.sprite_frames.get_frame_count("carrot"):
9         animated_sprite_2d.frame = animated_sprite_2d.frame + 1
8         print("plant has grown")
7
6 # 4. când planta este ștearsă putem accesa o metodă în clasa globală care primește tipul plantei
5 func _exit_tree() -> void:
4     # lasam carrot momentan, dar vom avea si alte tipuri de plante in viitor
3     PlayerData.add_plant("carrot")
2
1 func animate_on_ground() -> void:
18     pass

```

```

12 func _on_player_tool_used(tool: Player.Tool, pos: Vector2) → void:
11     var grid_pos: Vector2i = Vector2i(floor(pos.x / TILE_SIZE), floor(pos.y / TILE_SIZE))
10     match tool:
9         # 1. În primul rând să alegem un tool care să se ocupe de recoltare, asset pack-ul folosit
8         # nu dispune de un tool dedicat pentru această acțiune, astfel, putem folosi lopata(dig)
7         Player.Tool.DIG:
6             dirt_layer.set_cells_terrain_connect([grid_pos], 0, 0, false)
5             # 2. trebuie să intrăm în cazul dig(pentru lopată) și verificăm că există o plantă la
4             # coordonatele alese
3             if planted_cells.has(grid_pos):
2                 # 6. dacă în dicționar avem poziția grid-ului, putem șterge planta
1                 planted_cells[grid_pos].animate_on_ground()
37                # ștergem din dicționar cheia plantei
1                planted_cells.erase(grid_pos)

```



Pentru planta care se va afla scoasă din pământ ce credeți că ar trebui să adăugăm?(din punct de vedere vizual)



Avem două opțiuni, adăugăm un `sprite2D` nou cu o imagine care să arate întreaga plantă pentru când a fost recoltată din pământ sau adăugăm un ultim frame în `animatedsprite2D` pe care îl avem deja. Eu voi alege varianta a doua.

Am adăugat un ultim frame

Animations: Animation Frames:

5.0 FPS

Frame Duration: x 1.0

Filter Animations

carrot

0 1 2 3 4



Acum trebuie să implementăm metoda `animate_on_ground` din scriptul `plante.gd`.

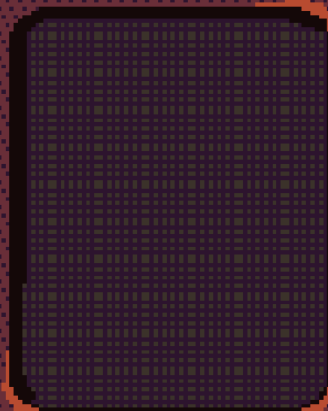
Ce aș vrea să se întâmple când sap planta din pământ?

Aș vrea să sară într-o direcție aleatorie pe o rază definită de mine.

Exercițiu: creați două variabile care să fie numere aleatorii și să reprezinte Offset-ul coordonatelor noi ale plantei unde va sări.

Puteți să căutați în documentație cum să generați numere random în Godot pentru ajutor.

În final, creați o variabilă care să reprezinte poziția finală a plantei săpate.



Creăm două numere aleatorii, apoi creăm poziția finală a plantei.

```
7 func animate_on_ground() → void:  
6     var x: float = randf_range(-12.0, 12.0)  
5     var y: float = randf_range(-12.0, 12.0)  
4     var final_pos: Vector2 = position + Vector2(x, y)
```

Mai rămâne să creăm un tween și să animăm proprietatea poziției.



Exercițiu: creați un tween și folosiți metoda `tween_property` care primește ca argumente: un nod(aici puteți să puneți `self`), numele unei proprietăți în ghilimele(este un string argumentul), valoarea finală a proprietății și durata animației.

(există exemple și în documentație)

```
7 func animate_on_ground() → void:  
6     var x: float = randf_range(-12.0, 12.0)  
5     var y: float = randf_range(-12.0, 12.0)  
4     var final_pos: Vector2 = position + Vector2(x, y)  
3  
2     var tween: Tween = get_tree().create_tween()  
1     tween.tween_property(self, "position", final_pos, 0.2)
```

Acum ar trebui să adăugăm niște constante pentru a nu lăsa numere magice în cod. Creați o constantă numită `RADIUS` cu valoarea 12.

```

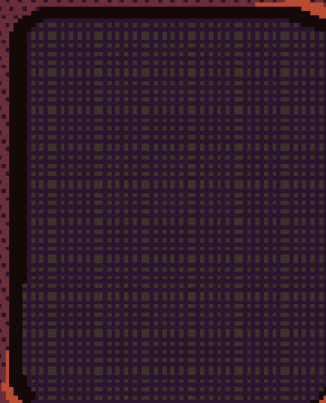
22 class_name Plant extends StaticBody2D
21
20 @onready var animated_sprite_2d: AnimatedSprite2D = $AnimatedSprite2D
19 var grid_pos: Vector2 = Vector2.ZERO
18 const RADIUS: float = 12
17
16 func grow_plant() -> void:
15     # check the frame is less than the frame count and then increment the frame
14     if animated_sprite_2d.frame < animated_sprite_2d.sprite_frames.get_frame_count("carrot"):
13         animated_sprite_2d.frame = animated_sprite_2d.frame + 1
12         print("plant has grown")
11
10 # 4. când planta este ştearsă putem accesa o metodă în clasa globală care primeşte
    tipul plantei
9 func _exit_tree() -> void:
8     # lasam carrot momentan, dar vom avea si alte tipuri de plante in viitor
7     PlayerData.add_plant("carrot")
6
5 func animate_on_ground() -> void:
4     var x: float = randf_range(-RADIUS, RADIUS)
3     var y: float = randf_range(-RADIUS, RADIUS)
2     var final_pos: Vector2 = position + Vector2(x, y)
1
23 var tween: Tween = get_tree().create_tween()
1 tween.tween_property(self, "position", final_pos, 0.2)

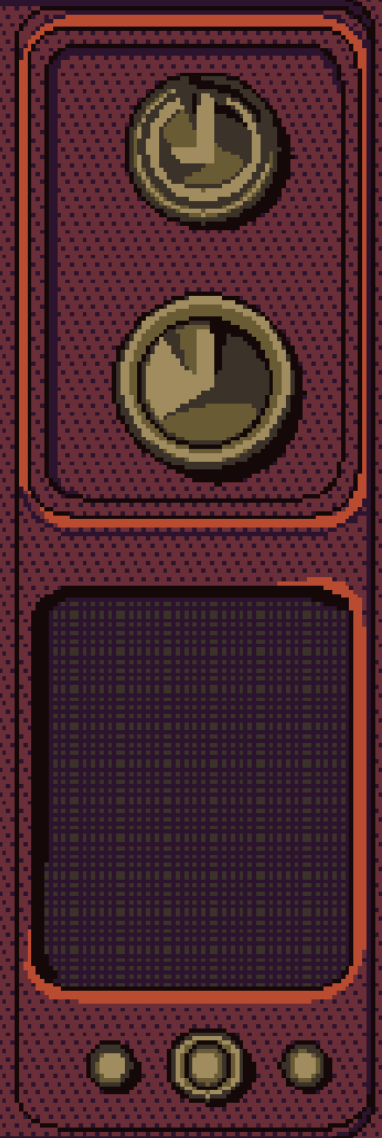
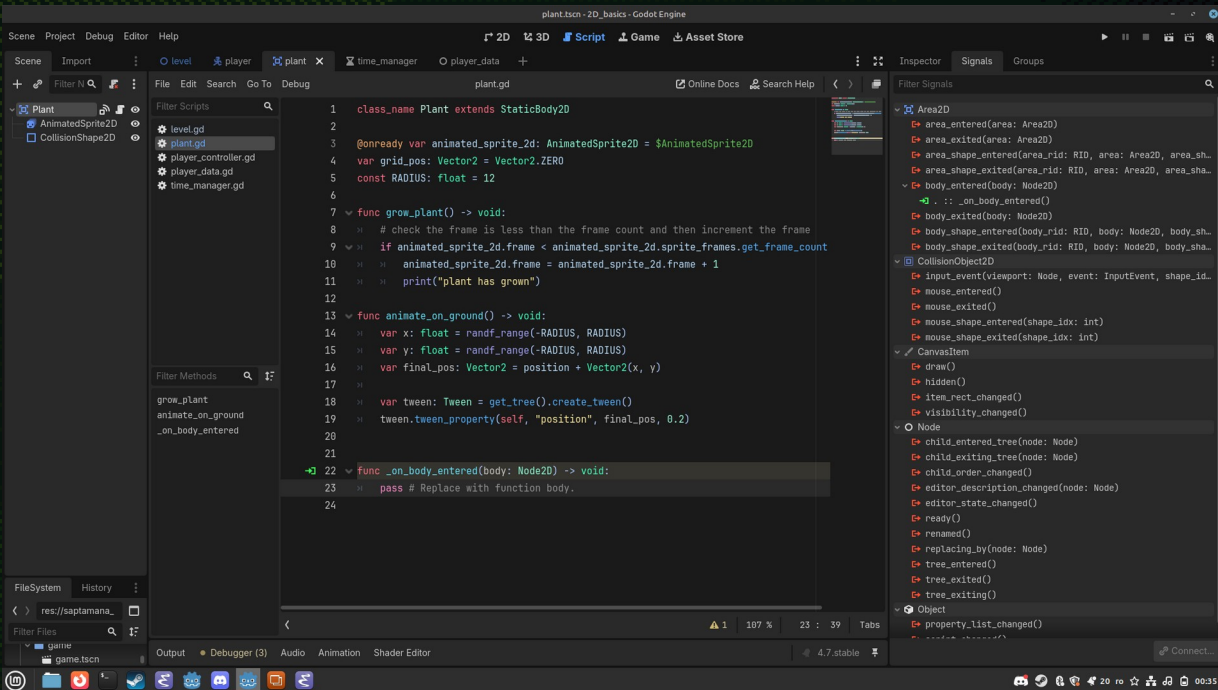
```

Putem elimina metoda `_exit_tree` deoarece nu mai avem nevoie de ea momentan. Acum putem să detectăm dacă planta este lovită de player și atunci să adăugăm planta în inventarul jucătorului.

Pentru a detecta că două sau mai multe corpuri se lovesc, godot pune la dispoziție un alt corp de fizică numit `Area2D` care dispune de mai multe semnale folositoare.

Exercițiu: Schimbați nodul `plant` din `staticbody2D` în `area2D`(dacă dați click dreapta pe nod aveți mai multe opțiuni). După ce ați schimbat nodul în tabul `signals` aveți la dispoziție mai multe semnale, cel pe care vrem să îl folosim este `body_entered`, dați double click pe el pentru a crea un callback.





În acest callback ar trebui să verificăm că body-ul care se lovește de plantă este de tip Player(vom avea mai multe characterbody2D npc-uri și nu vrem să ne fure plantele când le recoltăm).

Exercițiu: 1.verificați că body este player

2. creați un tween pe proprietatea scale care să aibă valoarea finală Vector2.ZERO și durata de 0.2 secunde.

3. la finalul tween-ului, apelați metoda din PlayerData pentru a adăuga planta în inventar apoi dați queue_free la nod pentru a îl șterge(pentru asta trebuie să așteptați cumva până se termină tween-ul).

```
12 func _on_body_entered(body: Node2D) → void:
11     # daca nu e player, iesim din metoda
10     if not body is Player:
9         return
8     # cream un tween
7     var tween: Tween = get_tree().create_tween()
6     # scalam obiectul la zero
5     tween.tween_property(self, "scale", Vector2.ZERO, 0.2)
4     await tween.finished
3     # adaug planta in inventar
2     PlayerData.add_plant("carrot")
1     queue_free()
```

```
switched tool to: DIG
switched tool to: CARRY
switched tool to: DIG
```

Ultimul pas este să setăm planta pe ultimul frame în metoda `animate_on_ground`.

Exercițiu: creați o variabilă numită `last frame` de tip `int` care să stocheze numărul de frame-uri - 1.

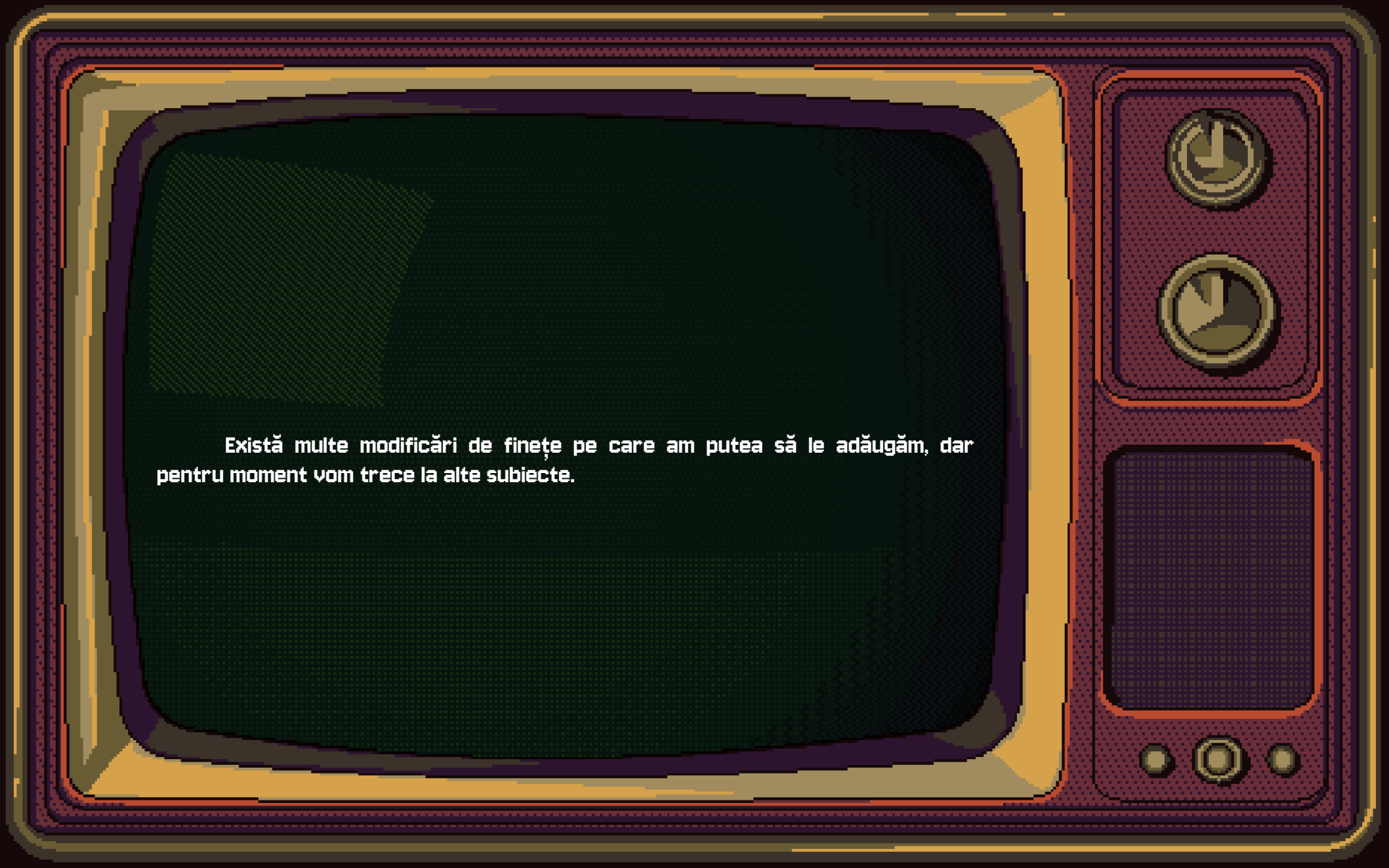
Hint: aveți o metodă care folosește deja asta.

În metoda `grow plant` modificați codul astfel încât să crească doar până la penultimul frame planta.

Get frame count returneaza numarul de frame-uri adica 5, dar frame-urile sunt numerotate de la 0 la 4, deci ultimul frame este de fapt 4 când vrem să îl accesăm din cod.

```
29 var last_frame: int
28 func _ready() → void:
27     last_frame = animated_sprite_2d.sprite_frames.get_frame_count("carrot") - 1
26
```

```
4 func grow_plant() → void:
3   # verific că frame-ul curent e mai mic decât penultimul frame
2   if animated_sprite_2d.frame < last_frame - 1:
1       animated_sprite_2d.frame = animated_sprite_2d.frame + 1
14  print("plant has grown")
```



Există multe modificări de finețe pe care am putea să le adăugăm, dar pentru moment vom trece la alte subiecte.

